Test Assessment: Swift, SwiftUI, Apollo GraphQL, Swift Package Manager

Objective

Create an iOS application that browses countries from a public GraphQL API, using Swift and SwiftUI with Apollo iOS for the network layer. The application should present a searchable list and a detail screen, structured across Swift packages, with the data layer designed so it can be tested without hitting the network.

The API is public and requires no authentication: https://countries.trevorblades.com

Requirements

List screen

- Display all countries: flag emoji, name, and continent.
- Filter the list by a search field matching country name.
- Debounce the search input so a query is not issued on every keystroke.
- Show distinct loading, empty-result, and error states.
- Tapping a country opens the detail screen.

Detail screen

- Display the country's name, flag emoji, capital, currency, languages, and continent.
- Show loading and error states.

Both screens

- *(Optional)* Support light and dark mode using semantic colours, no hardcoded colour values.
- *(Optional)* Support Dynamic Type, remaining usable at accessibility text sizes.

Technical Details

1. Modular structure

- Organise the code as at least two local Swift packages consumed by the app target so one for the data/network layer, one for the UI layer. The app target should contain only composition and entry point.
- The UI layer must not depend on Apollo-generated types directly: map them to your own domain models at the data layer boundary.
- Enable strict concurrency checking. The submission should build without concurrency warnings.

2. Networking

- Use Apollo iOS. Set up Apollo iOS with the schema downloaded from the API and code generation configured, then write the operations and the layer around them.

- Define GraphQL operations for the list and detail screens. Use a fragment for the fields shared between them.
- Expose the network layer behind a protocol so it can be substituted in tests.
- Use async/await. No completion handlers in your own API surface.
- Handle failures explicitly, a network error should surface as a user-facing error state, not a crash or a silent empty list.

3. State and views

- Use the Observation framework (@Observable) for the list and detail view models, with @MainActor applied appropriately.
- Use Combine for the search debounce.
- *(Optional)* Implement one custom ViewModifier and one custom EnvironmentValues entry, each used by more than one view. These should be things the code genuinely needs, not added to satisfy the requirement.
- Ensure list rows have stable identity across reloads and filtering.

4. Tests

- Write tests for the list view model covering: successful load, empty results, and a network failure, using a mocked network boundary.
- Write at least one test covering the mapping from the GraphQL response to your domain model.
- Use swift-testing or XCTest, either is fine. Tests should not perform network requests.

Submission

- Provide a link to a public GitHub repository containing the source code, with commit history intact.
- Include instructions in the README for how to build and run the application, including the Xcode version, iOS deployment target, and the simulator it was developed against.
- Include a short section in the README covering the key decisions you made, what you deliberately left out, and what you would change with more time.
- Ensure the code follows best practices for quality and structure, including configuration for SwiftFormat and/or SwiftLint.